OKINGDOM OF SAUDI ARABIA | JAZAN UNIVERSITY

COLLEGE OF ENGINEERING & COMPUTER SCIENCE

GROUP ASSIGNMENT 2 - 2025-2026

Mini Project Report

Title of the project: Schedule Generator

	Name student	Student ID
Group 5	Afnan Yahya Ogdi	202307421
	Reem Mohammed Badri	202305024
	Aryaf Mohammed Elaqi	202306972
	Arwa Abbas Qumayri	202306439

Course with code: Artificial Intelligence (COMP 441)

Instructor: Dr.Anne Anoop

Contents:

1. Introduction……..3
2. Problem Definition……...3
3. Algorithm Explanation…….....3
4. Project Details……..4
    - Tools & Libraries……4
    - Implementation Steps……...4
    - Code Explanation……4
5. Results & Screenshots……..5
6. Conclusion……..14
7. References……...…14

1. Introduction

This mini project presents the development of an intelligent Schedule Generator designed to help university students automatically create conflict-free weekly timetables. The system reads a CSV file containing sections, times, and days, then generates all possible valid schedules based on the student's selected subjects.

The project applies core Artificial Intelligence concepts—specifically state-space search, constraint satisfaction, and systematic backtracking—to explore combinations of class groups and eliminate time conflicts. Additionally, the system supports personalized preferences such as selecting a desired day off, which adds another constraint to the search space.

This work demonstrates how AI search algorithms can be used to solve real-world academic planning problems efficiently and accurately.

2. Problem Definition

Students often struggle with schedule construction because:

- Each course has multiple groups with different timings.
- Times may overlap between chosen groups.
- Manual trial-and-error consumes significant time.
- Some students prefer a specific day off, increasing the complexity.

Therefore, the problem can be modeled as a Constraint Satisfaction Problem (CSP), where:

- Variables = selected subjects
- Domain of each variable = available groups for that subject
- Constraints =
- No time overlap
- No conflicting days
- Optional constraint: selected day must remain free

The system must generate all valid assignments that satisfy all constraints.

3. Algorithm Explanation

The project uses a Backtracking Search Algorithm, which is one of the classical AI methods used for solving CSP problems.

Why Backtracking?

Backtracking is suitable because:

1. The problem requires exploring combinations of course groups.
2. At each step, decisions must satisfy constraints (time, day, conflict).
3. The search tree is finite but combinatorial, making systematic pruning important.
4. Backtracking avoids generating full invalid schedules by eliminating conflicts early.

How Backtracking Works in This Project

- The algorithm picks a subject.
- It tries each available group for that subject.
- Before accepting a selection, it applies a constraint-checking function:
- No lecture overlaps (time constraint)
- No repetition on the same day (conflict constraint)
- Optional "day off" constraint
- If a conflict is detected → the branch is abandoned immediately.
- If no conflict → continue exploring the next subject.
- The process repeats until:
- All subjects have assigned groups (valid schedule), or
- No valid combination exists.

This approach ensures all conflict-free schedules are found without exploring unnecessary states.

4. Project Details

4.1. Tools & Libraries

- Google Colab – execution environment
- Python
- Pandas – loading and filtering class data
- Datetime – time parsing and comparison
- Google Colab Files

4.2. Implementation Steps

1. Upload the CSV file containing course sections.
2. Enter course codes or names.
3. Select an optional day off.
4. The program loads all schedule options.
5. Backtracking is applied to generate all non-conflicting schedules.
6. The system prints the top results.

4.3. Code Explanation

parse_time()

Converts time strings ("HH:MM" or "HH:MM:SS") into comparable time objects.

times_overlap()

Checks if two time intervals overlap on the same day.

If (start1 < end2) and (start2 < end1) → a conflict exists.

generate_schedules()
Core function implementing backtracking.

Responsibilities:

- Grouping data by subject and section.
- Filtering options with or without the off-day constraint.
- Building schedules recursively.
- Eliminating conflicting groups before deeper exploration.

The function returns:

- All valid schedules.
- A flag indicating whether the day-off requirement was satisfied.

5. Results & Screenshots

The system displays:

- Total number of valid schedules
- Whether the selected off-day was successfully kept free
- Detailed listing of each schedule:
- Subject
- Group
- Day
- Start/end time

Example Output Interpretation

A typical result shows:

Found 12 schedules with Sunday OFF.

```
Schedule #1
CS417 - Application Devlopment | thu 13:00:00 - 13:50:00 (Group 2172)
CS417 - Application Devlopment | tue 14:00:00 - 14:50:00 (Group 2172)
CS417 - Application Devlopment | wed 14:00:00 - 15:50:00 (Group 2172)
CS4343 - Distributed copmuting | mon 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | wed 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | tue 17:00:00 - 18:50:00 (Group 2169)
CS452 - Cloud Computing | mon 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | thu 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | tue 12:00:00 - 13:50:00 (Group 420)
CS4533 - Data Science | mon 12:00:00 - 12:50:00 (Group 2175)
CS4533 - Data Science | thu 10:00:00 - 10:50:00 (Group 2175)
CS4533 - Data Science | tue 08:00:00 - 09:50:00 (Group 2175)
CS4413 - Artificial Intelligence | mon 09:00:00 - 09:50:00 (Group 2322)
CS4413 - Artificial Intelligence | thu 09:00:00 - 09:50:00 (Group 2322)
CS4413 - Artificial Intelligence | thu 14:00:00 - 15:50:00 (Group 2322)
```

This means:

- All chosen subjects appear once.
- No time overlaps exist.
- Sunday was kept free across all selected groups.

Strength of Output
The system generates all possible combinations (up to a limit) and prints the first five for quick review.

6. Discussion & Analysis

This project demonstrates the efficiency of backtracking for solving academic scheduling problems. Given the combinatorial nature of course offerings, brute force enumeration would be computationally expensive. However, constraint-based pruning significantly reduces the search space.

Additionally, by modeling the problem as a CSP, this project aligns directly with AI topics covered in COMP 441, including:

- Search algorithms
- Constraint propagation
- State-space reasoning
- Decision pruning

The ability to incorporate a "day off" preference shows flexibility in constraint handling.

7. Conclusion

This mini project successfully achieved its goal of generating conflict-free schedules based on student input. The system applies AI search principles—specifically backtracking and constraint evaluation—to solve a practical, real-world problem faced by university students.

The model can be improved by integrating:

- A GUI interface for easier interaction
- Optimization algorithms (e.g., Genetic Algorithms) for best schedule selection
- Support for complex university formats
- Automatic suggestion of the "best schedule" based on student preferences

Overall, the project demonstrates strong application of AI concepts and effective Python implementation.

- Full Python Code

```
import pandas as pd
from datetime import datetime
from google.colab import files

def parse_time(t):
    t = t.strip()
    try:
        return datetime.strptime(t, "%H:%M").time()
    except ValueError:
        return datetime.strptime(t, "%H:%M:%S").time()

def times_overlap(a_from, a_to, b_from, b_to):
    s1, e1 = parse_time(a_from), parse_time(a_to)
    s2, e2 = parse_time(b_from), parse_time(b_to)
    return (s1 < e2) and (s2 < e1)

def generate_schedules(df, subjects, off_day=None, limit=10):
    choices = [] #Preparing Lecture Options
    for subj in subjects:
        options = df[
            (df['subject'].str.strip().str.lower() == subj.strip().lower()) |
            (df['subject_name'].str.strip().str.lower() == subj.strip().lower())
        ]
        groups = options.groupby('group')
        group_list = []
        for g, rows in groups:
            group_list.append(rows.to_dict('records'))
        if off_day:
```

```
            group_no_off = [g for g in group_list if all(r['day'].strip().lower() !=
off_day.strip().lower() for r in g)]
            choices.append({'subject': subj, 'no_off': group_no_off, 'all':
group_list})
        else:
            choices.append({'subject': subj, 'all': group_list})
    schedules = []
    #Backtracking Algorithm to Build Schedules
    def backtrack(i, current, use_no_off):
        nonlocal schedules
        if len(schedules) >= limit:
            return
        if i == len(choices):
            schedules.append([r for group in current for r in group])  # flatten
            return
        entry = choices[i]
        options = entry['no_off'] if (off_day and use_no_off) else entry['all']
        for group in options:
            conflict = False
            for g in group:
                for c in current:
                    for c_r in c:
                        if c_r['day'].lower() == g['day'].lower() and times_overlap(
                            c_r['time_from'], c_r['time_until'], g['time_from'],
g['time_until']):
                            conflict = True
                            break
                    if conflict:
                        break
```

```
            if conflict:
                break
        if not conflict:
            current.append(group)
            backtrack(i + 1, current, use_no_off)
            current.pop()
    #Handling Optional Day Off
    if off_day:
        backtrack(0, [], True)
        if schedules:
            return schedules, True
        schedules = []
        backtrack(0, [], False)
        return schedules, False
    else:
        backtrack(0, [], False)
        return schedules, False

print("Please upload your schedule CSV file:")
uploaded = files.upload()
file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)
print("\nFile loaded successfully.")
print(df.head(), "\n")

user_input = input("Enter the subjects or course codes (comma
separated):\n> ").strip()
```

```
wanted_subjects = [x.strip() for x in user_input.split(",") if x.strip()]

off_day = input("\nDo you want a day off? (like: sun) or leave empty if
not:\n> ").strip()
off_day = off_day if off_day else None



print("\nGenerating all possible non-conflicting schedules...")
schedules, off_ok = generate_schedules(df, wanted_subjects,
off_day=off_day)

if not schedules:
    print("No possible schedule found for these subjects.")
else:
    if off_day:
        if off_ok:
            print(f"\nFound {len(schedules)} schedules with {off_day} OFF.")
        else:
            print(f"\nNo schedules with {off_day} off, but found
{len(schedules)} possible schedules.")
    else:
        print(f"\nFound {len(schedules)} possible schedules.")



    for i, sched in enumerate(schedules[:5], 1):
        print(f"\nSchedule #{i}")
        for row in sched:
            print(f"{row['subject']} - {row['subject_name']} | "
```

f"{row['day']} {row['time_from']} - {row['time_until']} (Group {row['group']})")

- ScreanShots

Importing Required Libraries

[ ]

```
import pandas as pd
from datetime import datetime
from google.colab import files
```

Parsing Time Strings

[1]
0s

```
def parse_time(t):
    t = t.strip()
    try:
        return datetime.strptime(t, "%H:%M").time()
    except ValueError:
        return datetime.strptime(t, "%H:%M:%S").time()
```

Checking if Two Time Intervals Overlap

[2]
0s

```
def times_overlap(a_from, a_to, b_from, b_to):
    s1, e1 = parse_time(a_from), parse_time(a_to)
    s2, e2 = parse_time(b_from), parse_time(b_to)
    return (s1 < e2) and (s2 < e1)
```

Generating Non-Conflicting Schedules

[5]
✓ 0 s

```python
def generate_schedules(df, subjects, off_day=None, limit=10):
    choices = [] #Preparing Lecture Options
    for subj in subjects:
        options = df[
            (df['subject'].str.strip().str.lower() == subj.strip().lower()) |
            (df['subject_name'].str.strip().str.lower() == subj.strip().lower())
        ]
        groups = options.groupby('group')
        group_list = []
        for g, rows in groups:
            group_list.append(rows.to_dict('records'))
        if off_day:
            group_no_off = [g for g in group_list if all(r['day'].strip().lower() != off_day.strip().lower() for r in g)]
            choices.append({'subject': subj, 'no_off': group_no_off, 'all': group_list})
        else:
            choices.append({'subject': subj, 'all': group_list})
    schedules = []
    #Backtracking Algorithm to Build Schedules
    def backtrack(i, current, use_no_off):
        nonlocal schedules
        if len(schedules) >= limit:
            return
        if i == len(choices):
            schedules.append([r for group in current for r in group])  # flatten
            return
        entry = choices[i]
        options = entry['no_off'] if (off_day and use_no_off) else entry['all']
        for group in options:
            conflict = False
            for g in group:
                for c in current:
                    for c_r in c:
                        if c_r['day'].lower() == g['day'].lower() and times_overlap(
                            c_r['time_from'], c_r['time_until'], g['time_from'], g['time_until']):
                            conflict = True
                            break
                    if conflict:
                        break
                if conflict:
                    break
            if not conflict:
                current.append(group)
                backtrack(i + 1, current, use_no_off)
                current.pop()
    #Handling Optional Day Off
    if off_day:
        backtrack(0, [], True)
        if schedules:
            return schedules, True
        schedules = []
        backtrack(0, [], False)
        return schedules, False
    else:
        backtrack(0, [], False)
        return schedules, False
```

User Interface for File Upload

[8]
1 min

```python
print("Please upload your schedule CSV file:")
uploaded = files.upload()
file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)
print("\nFile loaded successfully.")
print(df.head(), "\n")
```

```
Please upload your schedule CSV file:
اختيار ملفات schedule.csv
schedule.csv(text/csv) - 6066 bytes, last modified: 11100% - 2025/11/ done
Saving schedule.csv to schedule.csv

File loaded successfully.
   group subject            subject_name  day time_from time_until
0   2172   CS417  Application Devlopment  thu  13:00:00   13:50:00
1   2172   CS417  Application Devlopment  tue  14:00:00   14:50:00
2   2172   CS417  Application Devlopment  wed  14:00:00   15:50:00
3   2186   CS417  Application Devlopment  sun  14:00:00   14:50:00
4   2186   CS417  Application Devlopment  mon  15:00:00   15:50:00
```

User Input for Subjects and Day Off

[16]

```python
user_input = input("Enter the subjects or course codes (comma separated):\n> ").strip()
wanted_subjects = [x.strip() for x in user_input.split(",") if x.strip()]

off_day = input("\nDo you want a day off? (like: sun) or leave empty if not:\n> ").strip()
off_day = off_day if off_day else None
```

```
Enter the subjects or course codes (comma separated):
> CS417,CS4343,CS452,CS4
```

User Input for Subjects and Day Off

[16]

```python
user_input = input("Enter the subjects or course codes (comma separated):\n> ").strip()
wanted_subjects = [x.strip() for x in user_input.split(",") if x.strip()]

off_day = input("\nDo you want a day off? (like: sun) or leave empty if not:\n> ").strip()
off_day = off_day if off_day else None
```

```
Enter the subjects or course codes (comma separated):
> CS417,CS4343,CS452,CS4533,CS4413

Do you want a day off? (like: sun) or leave empty if not:
>
```

User Input for Subjects and Day Off

[9]
20 s

```python
user_input = input("Enter the subjects or course codes (comma separated):\n> ").strip()
wanted_subjects = [x.strip() for x in user_input.split(",") if x.strip()]

off_day = input("\nDo you want a day off? (like: sun) or leave empty if not:\n> ").strip()
off_day = off_day if off_day else None
```

```
Enter the subjects or course codes (comma separated):
> CS417,CS4343,CS452,CS4533,CS4413

Do you want a day off? (like: sun) or leave empty if not:
> sun
```

Generate Schedules and Display Results

[10]
✓ 0s

```python
print("\nGenerating all possible non-conflicting schedules...")
schedules, off_ok = generate_schedules(df, wanted_subjects, off_day=off_day)

if not schedules:
    print("No possible schedule found for these subjects.")
else:
    if off_day:
        if off_ok:
            print(f"\nFound {len(schedules)} schedules with {off_day} OFF.")
        else:
            print(f"\nNo schedules with {off_day} off, but found {len(schedules)} possible schedules.")
    else:
        print(f"\nFound {len(schedules)} possible schedules.")
```

```
Generating all possible non-conflicting schedules...

Found 10 schedules with sun OFF.
```

Display Top Schedules

[11]
✓ 0 s

```python
for i, sched in enumerate(schedules[:5], 1):
    print(f"\nSchedule #{i}")
    for row in sched:
        print(f"{row['subject']} - {row['subject_name']} | "
              f"{row['day']} {row['time_from']} - {row['time_until']} (Group {row['group']})")
```

```
Schedule #1
CS417 - Application Devlopment | thu 13:00:00 - 13:50:00 (Group 2172)
CS417 - Application Devlopment | tue 14:00:00 - 14:50:00 (Group 2172)
CS417 - Application Devlopment | wed 14:00:00 - 15:50:00 (Group 2172)
CS4343 - Distributed copmuting | mon 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | wed 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | tue 17:00:00 - 18:50:00 (Group 2169)
CS452 - Cloud Computing | mon 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | thu 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | tue 12:00:00 - 13:50:00 (Group 420)
CS4533 - Data Science | mon 12:00:00 - 12:50:00 (Group 2175)
CS4533 - Data Science | thu 10:00:00 - 10:50:00 (Group 2175)
CS4533 - Data Science | tue 08:00:00 - 09:50:00 (Group 2175)
CS4413 - Artificial Intelligence | mon 09:00:00 - 09:50:00 (Group 2322)
CS4413 - Artificial Intelligence | thu 09:00:00 - 09:50:00 (Group 2322)
CS4413 - Artificial Intelligence | thu 14:00:00 - 15:50:00 (Group 2322)

Schedule #2
CS417 - Application Devlopment | thu 13:00:00 - 13:50:00 (Group 2172)
CS417 - Application Devlopment | tue 14:00:00 - 14:50:00 (Group 2172)
CS417 - Application Devlopment | wed 14:00:00 - 15:50:00 (Group 2172)
CS4343 - Distributed copmuting | mon 18:00:00 - 18:50:00 (Group 2198)
CS4343 - Distributed copmuting | tue 18:00:00 - 18:50:00 (Group 2198)
CS4343 - Distributed copmuting | thu 18:00:00 - 19:50:00 (Group 2198)
CS452 - Cloud Computing | mon 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | thu 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | tue 12:00:00 - 13:50:00 (Group 420)
CS4533 - Data Science | mon 12:00:00 - 12:50:00 (Group 2175)
CS4533 - Data Science | thu 10:00:00 - 10:50:00 (Group 2175)
CS4533 - Data Science | tue 08:00:00 - 09:50:00 (Group 2175)
CS4413 - Artificial Intelligence | mon 09:00:00 - 09:50:00 (Group 2322)
CS4413 - Artificial Intelligence | thu 09:00:00 - 09:50:00 (Group 2322)
CS4413 - Artificial Intelligence | thu 14:00:00 - 15:50:00 (Group 2322)

Schedule #3
CS417 - Application Devlopment | mon 11:00:00 - 11:50:00 (Group 2212)
CS417 - Application Devlopment | tue 08:00:00 - 09:50:00 (Group 2212)
CS417 - Application Devlopment | tue 10:00:00 - 11:50:00 (Group 2212)
CS4343 - Distributed copmuting | mon 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | wed 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | tue 17:00:00 - 18:50:00 (Group 2169)
CS452 - Cloud Computing | mon 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | thu 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | tue 12:00:00 - 13:50:00 (Group 420)
CS4533 - Data Science | wed 15:00:00 - 15:50:00 (Group 2247)
CS4533 - Data Science | thu 15:00:00 - 15:50:00 (Group 2247)
CS4533 - Data Science | thu 12:00:00 - 13:50:00 (Group 2247)
CS4413 - Artificial Intelligence | tue 15:00:00 - 15:50:00 (Group 2249)
CS4413 - Artificial Intelligence | thu 14:00:00 - 14:50:00 (Group 2249)
CS4413 - Artificial Intelligence | mon 12:00:00 - 13:50:00 (Group 2249)

Schedule #4
CS417 - Application Devlopment | mon 11:00:00 - 11:50:00 (Group 2212)
CS417 - Application Devlopment | tue 08:00:00 - 09:50:00 (Group 2212)
CS417 - Application Devlopment | tue 10:00:00 - 11:50:00 (Group 2212)
CS4343 - Distributed copmuting | mon 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | wed 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | tue 17:00:00 - 18:50:00 (Group 2169)
CS452 - Cloud Computing | mon 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | thu 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | tue 12:00:00 - 13:50:00 (Group 420)
CS4533 - Data Science | wed 15:00:00 - 15:50:00 (Group 2247)
CS4533 - Data Science | thu 15:00:00 - 15:50:00 (Group 2247)
CS4533 - Data Science | thu 12:00:00 - 13:50:00 (Group 2247)
CS4413 - Artificial Intelligence | mon 13:00:00 - 13:50:00 (Group 2313)
CS4413 - Artificial Intelligence | wed 13:00:00 - 13:50:00 (Group 2313)
CS4413 - Artificial Intelligence | thu 10:00:00 - 11:50:00 (Group 2313)

Schedule #5
CS417 - Application Devlopment | mon 11:00:00 - 11:50:00 (Group 2212)
CS417 - Application Devlopment | tue 08:00:00 - 09:50:00 (Group 2212)
CS417 - Application Devlopment | tue 10:00:00 - 11:50:00 (Group 2212)
CS4343 - Distributed copmuting | mon 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | wed 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | tue 17:00:00 - 18:50:00 (Group 2169)
CS452 - Cloud Computing | thu 14:00:00 - 14:50:00 (Group 2358)
CS452 - Cloud Computing | tue 14:00:00 - 14:50:00 (Group 2358)
CS452 - Cloud Computing | mon 14:00:00 - 15:50:00 (Group 2358)
CS4533 - Data Science | wed 15:00:00 - 15:50:00 (Group 2247)
CS4533 - Data Science | thu 15:00:00 - 15:50:00 (Group 2247)
CS4533 - Data Science | thu 12:00:00 - 13:50:00 (Group 2247)
CS4413 - Artificial Intelligence | mon 13:00:00 - 13:50:00 (Group 2313)
CS4413 - Artificial Intelligence | wed 13:00:00 - 13:50:00 (Group 2313)
CS4413 - Artificial Intelligence | thu 10:00:00 - 11:50:00 (Group 2313)
```

8. References

- Python Docs: https://docs.python.org
- Pandas: https://pandas.pydata.org
- Google Colab: https://colab.research.google.com